COMPUTER ARCHITECTURE

Week 4: From C to RV32I

VNU - UNIVERSITY *of* ENGINEERING *&* TECHNOLOGY

Agenda

1. **Introduction**
   The 3-step translation method
2. **Control Flow**
   if/else → branches · while/for → branch + jump
3. **Arrays & Pointers**
   Indexing vs pointer iteration · base + offset · instruction cost
4. **Procedures & Stack**
   Memory layout · stack frames · calling convention · recursion
5. **Translation & Startup**
   Compile → assemble → link → load · end-to-end trace

01

Introduction

From High-Level C to RV32I

Learning Objectives

By the end of this lecture, you will be able to map high-level C constructs to their corresponding RV32I assembly patterns.

C construct	RISC-V mechanism	Key instruction(s)	Watch out for
if / else	Branches	beq, bne, blt, bge	Invert the condition — branch to rare path, fall through common path
loops	Branch + jump	bne + j (= jal x0)	Every loop compiles to the same goto-with-label pattern
arrays	Base + offset	slli, add, lw/sw	Indexing requires multiply — pointer iteration avoids it
functions	jal + stack	jal, jalr, sw/lw sp	Recursion overwrites ra — callee must save ra before calling further
programs	Compile → link → load	(toolchain stages)	"compile" colloquially means all three stages combined

From High-Level C to RV32I

RV32I supports only integer data types and provides ~40 ***base*** instructions, with no complex or specialized operations → How does a compiler translate high-level language constructs into RV32I code?

C construct	RISC-V mechanism
`if / else`	`branches`
`loops`	`branches + jumps`
`arrays`	`base + offset`
`functions`	`jal + stack`
`programs`	`compile → link → load`

Key insight: We use this 3-step scaffold for every C construct — control flow, arrays, and functions so the translation pattern becomes automatic.

The 3-Step Translation Method

1 Write the C construct

e.g. `if (i == j) f = g + h;`

2 Rewrite with goto

bne-exit pattern — makes the branch explicit

3 Translate to assembly

Replace `goto` → `j/jal`, conditions → `beq/bne`

02

Control Flow

if / else · while · for

If Statement: 3-step Translation

Step 1: C code

```
if (i == j)
  f = g + h;

Register mapping:
f→x12   g→x13   h→x14
i→x15   j→x16
```

Step 2: Rewrite condition (invert)

```
// invert == to !=
if (i != j) goto Exit;
  f = g + h;
Exit:
```

Step 3: RV32I assembly

```
bne x15, x16, Exit
add x12, x13, x14
Exit:
```

Why invert the condition? Branch to the rare path, fall through the common path. This improves branch-predictor accuracy and instruction-cache locality. Compilers always prefer the fall-through form.

Extension: if / else

```
if (i == j)
  f = g + h;
else
  f = g - h;
```

```
if (i != j) goto Else;
  f = g + h;
  goto Exit;
Else:
  f = g - h;
Exit:
```

```
bne x15, x16, Else
add x12, x13, x14
j Exit
Else:
sub x12, x13, x14
Exit:
```

Note: `j` is a pseudoinstruction - it expands to `jal x0, label`. The assembler replaces it; the CPU never sees `j`.

Loops

RV32I has no loop instruction → Every loop compiles to: conditional branch + unconditional jump.

Universal pattern: Every C while, do-while, and for loop has the same decomposition pattern

```
for (init; cond; update) body;
// identical to:
init;  while (cond) { body; update; }
// Compilers generate the same goto pattern – init once, then: label / test / body / update / jump.
```

Example

Step 1: C code

```
while (j == k)
  i = i + 1;

Register mapping:
i → x13   j → x14   k → x15
```

Step 2: Rewrite condition (invert)

```
Loop:
  if (j != k)
    goto Exit;
  i = i + 1;
  goto Loop;
Exit:
```

Step 3: RV32I assembly

```
Loop:
  bne x14,x15,Exit
  addi x13,x13,1
  j Loop
Exit:
```

03

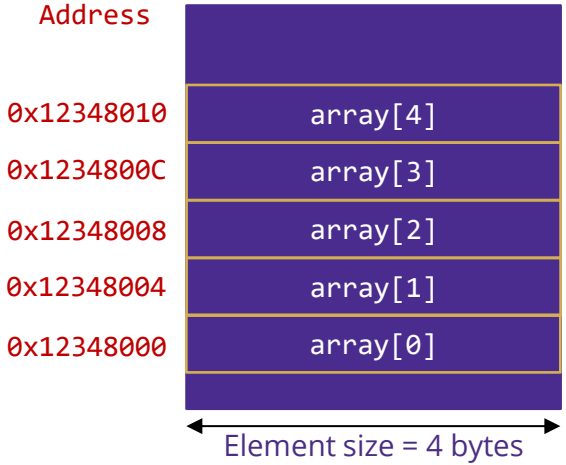
Arrays & Pointers

Indexing · pointer iteration · instruction cost comparison

Arrays in C — Memory Layout

An array in C is used to store many values of the same type in contiguous memory, e.g. `int A[40];`

- No dedicated array instruction in RV32I - the compiler uses two strategies: *indexing* or *pointer iteration*.



Method 1 — Array Indexing: A[i]

Address(A[i]) = base + i × element_size
Compiler emits: `slli` (to compute `i×4`) + `add` + `lw`.
Element size depends on type — always 4 bytes for `int`.

Method 2 — Pointer Arithmetic: *(A + i)

Move a pointer forward by element_size bytes each iteration.
Compiler emits: `addi ptr, ptr, 4` + `lw`.
Eliminates the multiply — faster for sequential access.

Note: In C, `A[i]` is syntactic sugar for `*(A+i)`. The two methods produce different assembly but identical results.

Array Indexing Approach — Assembly

Example: count zeros in a 40-element int array. Register map: A[]→t0 result→t6 i→t1

C code

```
result = 0;
i = 0;
while (i < 40) {
  if (A[i] == 0)
    result++;
  i++;
}
```

RV32I assembly

```
addi t6,zero,0  # result=0
addi t1,zero,0  # i=0
addi t2,zero,40 # end=40
loop:
  bge t1,t2,end # i>=40?
  slli t3,t1,2  # t3=i*4
  add  t4,t0,t3 # &A[i]
  lw   t5,0(t4) # t5=A[i]
  bne  t5,zero,skip
  addi t6,t6,1  # result++
skip:
  addi t1,t1,1  # i++
  j loop
end:
```

Type note:

slli by 2 (×4) is correct only for int. For long use slli by 3. Always tie the shift to the element type.

Instruction count per iteration:

slli + add + lw + conditional branch + addi (i++) + j = 6 instructions overhead before doing any useful work

Pointer Iteration Approach

Same task: But replace index arithmetic with pointer increment

Pointer assembly (A[]→t0 result→t6 &A[i]→t1)

```
addi t6,zero,0   # result=0
addi t1,t0,0     # ptr=&A[0]
addi t2,t0,160   # end=&A[40]
                 # (40 × 4 = 160)
loop:
  bge t1,t2,end  # ptr>=end?
  lw  t3,0(t1)   # t3=*ptr
  bne t3,zero,skip
  addi t6,t6,1   # result++
skip:
  addi t1,t1,4   # ptr+=4
  j loop
end:
```

Side-by-side comparison

	Indexing	Pointer
Addr calc	slli + add + lw	lw only
Instr/iter	6 overhead	4 overhead
Savings	—	−2 per iteration
Code size	14 lines	12 lines
Readability	A[i] familiar	pointer-fluent

Efficiency verdict:

Pointer iteration is more efficient for sequential access — saves one shift and one add per iteration.
For random access, indexing is unavoidable.

04

Procedures & Stack

Memory layout · stack frames · calling convention · recursion

Memory Layout

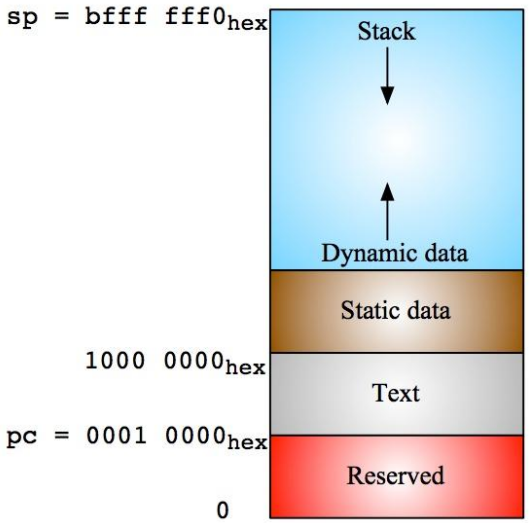
To implement procedure calls we first need to understand how memory is organised at runtime.

Text segment
Binary instruction codes — your compiled program. Read-only at runtime.

Static data segment
Global variables declared once per program. The gp (global pointer) register points here.

Dynamic data (heap)
Variables from malloc(). Grows upward toward the stack.

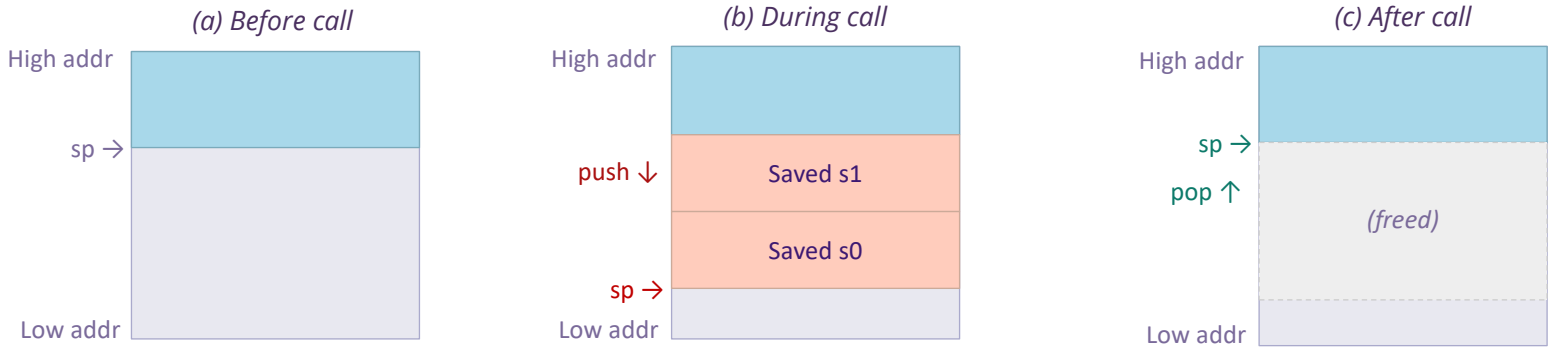
Stack segment
Saved registers, local variables, function arguments beyond a0–a7. Grows downward from high memory. sp (x2) tracks the top.



Stack Management with RV32I

Need stack to save old values before calling a procedure, then restore them on return

- RISC-V convention: sp (x2) is the stack pointer. Push decrements sp; pop increments sp.



Push and pop pattern

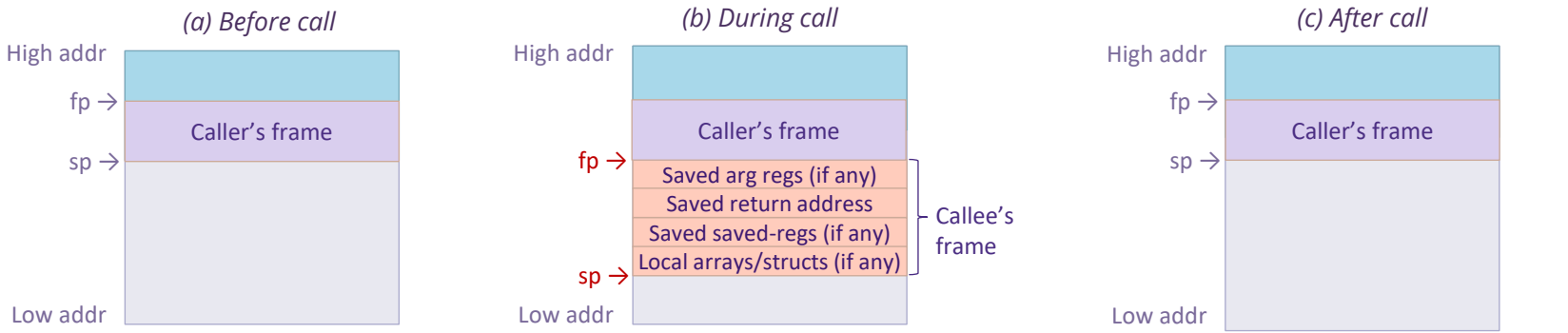
```
# PUSH: allocate and save
addi sp, sp, -8   # grow stack
sw   s1, 4(sp)    # save s1
sw   s0, 0(sp)    # save s0
```

```
# POP: restore and deallocate
lw   s0, 0(sp)    # restore s0
lw   s1, 4(sp)    # restore s1
addi sp, sp, 8    # shrink stack
```

Allocating Space on the Stack

Local data is allocated by the callee in a procedure frame (also called activation record)

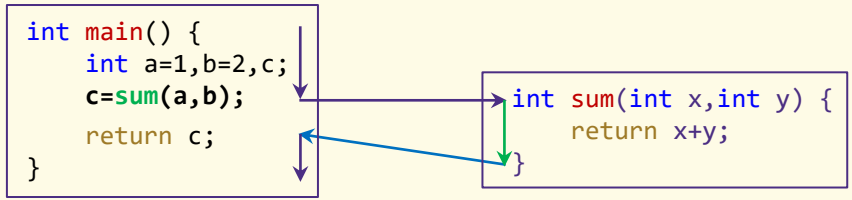
- Frame Pointer (fp / s0): optional stable reference - anchors the frame so sp can move freely during execution.



Frame region	Contents	Who owns it
Saved arg regs	a0–a7 values caller passed (if callee modifies them)	Callee saves if needed
Return address	ra — where to jump when function returns	Callee must save before any nested call
Saved registers	s0–s11 values callee uses	Callee saves/restores (preserved contract)
Local data	Arrays, structs, spilled locals	Callee allocates

Procedures — Introduction

```
int main() {
  int a=1, b=2, c;
  c = sum(a, b);   // call sum
  return c;
}
int sum(int x, int y) { return x + y; }
```



Procedure: a defined segment of code executed via a function call to achieve a designated task.

- C procedures support parameters, return values, and recursion — all implemented with jumps and the stack.

Six fundamental steps in calling a procedure

1. Put arguments in registers
2. Transfer control to the procedure
3. Allocate stack frame
4. Perform tasks of the procedure
5. Place return value & release local storage
6. Return control to the caller

RV32I Procedure Call Convention

Calling convention: a uniform contract for how machine resources transfer control and data between procedures.

Step	Description	RISC-V registers	Notes
1	Put arguments in registers	a0–a7 (x10–x17)	Up to 8 args; excess go on the stack
2	Transfer control to procedure	j, jal, jalr	jal saves PC+4 into ra automatically
3	Acquire local storage	sp, fp (x2, x8)	Decrement sp; optionally anchor fp to old sp
4	Perform task; allocate work registers	t0–t6, s0–s11	t regs: non-preserved. s regs: callee-saved
5	Place return value(s) in register	a0, a1	64-bit returns use a0 (low) + a1 (high)
6	Restore stack & return to caller	jr ra (= jalr x0,ra,0)	ra holds the return address

Preserved (callee-saved): caller can rely on these

```
sp, gp, tp, fp (s0), s1–s11
(callee must save+restore if used)
```

Non-preserved (caller-saved): may be overwritten by callee

```
a0–a7, ra, t0–t6
(caller must save if needed across a call)
```

ra is non-preserved — the callee may overwrite it. Before making any nested call, save ra to the stack. (See recursion slides.)

Passing Control — From Manual to jal

Manual approach — 2 instructions

```
1008  addi ra, zero, 1016  # ra = return address (hardcoded — only works if 1016 fits in 12-bit imm!)
1012  j sum                # jump to sum (pseudoinstruction for jal x0, sum)
1016  ...                  # next instruction
```

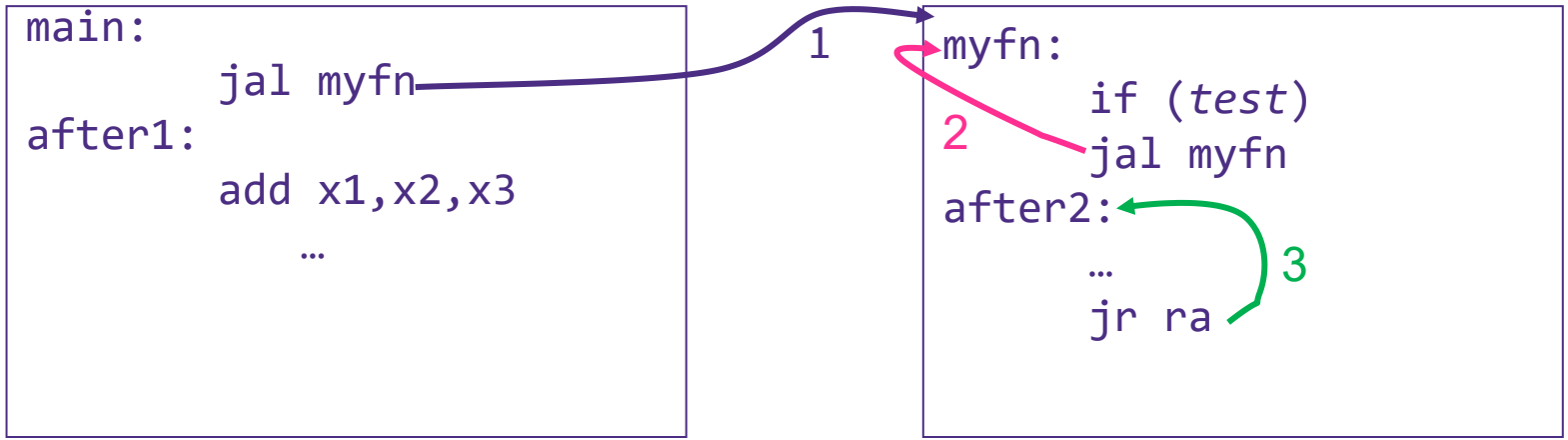
Limitation: addi can only encode a 12-bit signed immediate (±2048). Hardcoding the return address fails for functions far apart in memory.

Better — jal does both in one instruction:

```
1008  jal sum   # ra = PC+4 = 1012,  goto sum  — single instruction, 20-bit offset
1012  ...       # next instruction  (this is what ra holds)
```

jal precision: `jal rd, label` stores PC+4 (address of next instruction) into `rd`, then jumps to `label`. With `rd=ra` this is the call instruction. With `rd=x0` this is an unconditional jump (`j` pseudoinstruction).

Problem with Recursion



Solution: Save `ra` to the stack before making any nested call. Restore it before returning. This is exactly what the push/pop sequence on the next slide does.

The problem: When `myfn` calls itself, `jal` overwrites `ra` with the new return address. The original ra (pointing back to main) is lost. The function cannot return to its caller.

Nested Procedures — Register Saving Strategy

> RISC-V divides registers into two classes to minimize expensive stack traffic:

Class	Registers	Who saves?	Guarantee
Preserved (callee-saved)	sp, gp, tp, fp(s0), s1–s11	Callee saves before use, restores before return	Caller sees same value after the call
Non-preserved (caller-saved)	a0–a7, t0–t6	Caller saves before call if it needs the value later	Callee may freely overwrite these

How this reduces stack traffic

Leaf function (makes no calls)

Use t0–t6 for all work. Nothing needs saving - t regs are non-preserved, so the caller already knows they may change.

Non-leaf function (makes calls)

Use s registers for values that must survive across calls. Save them once on entry, restore on exit. Saves only what you use.

Recursive function

Must save ra (overwritten by jal) and any s registers you use. Use a0–a7 for arguments, which are non-preserved and need no save.

Compiling a Nested Call — sumSquare

```
int sumSquare(int x, int y) {
  return mult(x, x) + y; }
```

Register mapping: x → a0 y → a1 return value → a0

sumSquare is a non-leaf: it calls `mult` → must save `ra` and `a1` (y) before the call.

Assembly — sumSquare:

```
sumSquare:
  addi sp, sp, -8  # allocate 2 words on stack
  sw   ra, 4(sp)   # SAVE ra (will be overwritten by jal mult)
  sw   a1, 0(sp)   # SAVE y (a1 non-preserved: mult may overwrite)

  mv   a1, a0      # second arg = x (mult(x, x))
  jal  mult        # call mult — ra now = return addr inside sumSquare

  lw   a1, 0(sp)   # RESTORE y
  add  a0, a0, a1  # result = mult(x,x) + y
  lw   ra, 4(sp)   # RESTORE ra (now points back to sumSquare's
caller)
  addi sp, sp, 8   # deallocate frame
  jr   ra          # return to original caller
mult: ...
```

PUSH — save before nested call

- ra: non-preserved, will be overwritten by jal mult
- a1 (y): non-preserved, mult is free to change it

POP — restore after nested call

- Restore y (a1) so we can compute mult()+y
- Restore ra so jr ra goes to OUR caller, not mult's return

Rule: Save before call. Restore after. In reverse order. Do not save what you don't use.

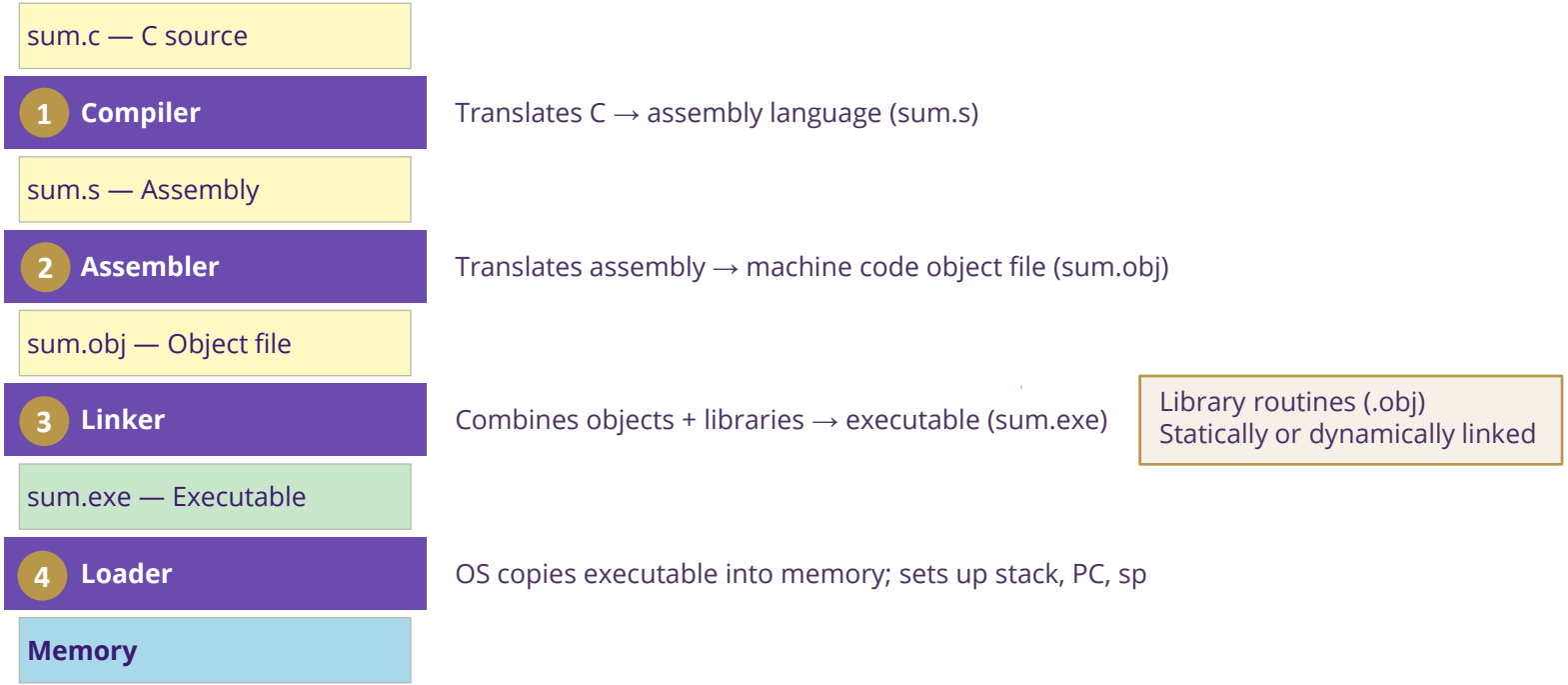
05

Translation & Startup

Compile · Assemble · Link · Load

From Writing to Running a Program

When most people say "compile" they mean the whole pipeline: compile + assemble + link.



Stage 3 — Linker

> Combines several object files into a single executable — resolves all cross-file references.

- Merges text segments, merges data segments — 'stitches' routines together
- Resolves symbol addresses using the symbol table and relocation info from each .obj
- Patches location-dependent and external references with correct final addresses

> Static linking vs dynamic linking

	Static linking	Dynamic linking (DLL / .so)
When	At link time — library code is copied into the executable	At load or run time — library loaded separately
Binary	Larger — includes all library code	Smaller — references only
Updates	Requires full recompile when library changes	Uses updated library automatically on next run
Cost	Faster startup — everything is pre-resolved	Small runtime overhead for symbol lookup
Use case	Embedded, standalone tools	Desktop/server apps, OS shared libraries

Executable file: An object file with ALL references resolved. A relocating loader can place it at any virtual address - simplified by virtual memory.

Linking Worked Example — Tracing a jal Offset

Two object files: A.obj (text size 0x100) and B.obj (text size 0x200). Text segment loads at 0x0004_0000.

A.obj section	Local addr	Instruction
Text[0x0]	0	lw x10, 0(x3)
Text[0x4]	4	jal x1, B.func ← unresolved
Data[0x0]	—	(X)
Symbol X	—	undefined
Symbol B	—	undefined

B.obj section	Local addr	Instruction
Text[0x0]	0	sw x11, 0(x3)
Text[0x4]	4	jal x1, A.fn ← unresolved
Data[0x0]	—	(Y)
Symbol Y	—	undefined
Symbol A	—	undefined

Linker resolution — how the jal offset is calculated

After linking, A.obj text is placed at: 0x0004_0000
After linking, B.obj text is placed at: 0x0004_0100 (= 0x0004_0000 + A text size 0x100)
A's jal at address 0x0004_0004 targets B.func at 0x0004_0100:
offset = target - PC = 0x0004_0100 - 0x0004_0004 = 0xFC = 252 (decimal)
→ linker patches A's jal to: `jal x1, 252`
B's jal at address 0x0004_0104 targets A.fn at 0x0004_0000:
offset = 0x0004_0000 - 0x0004_0104 = -0x104 = -260 (decimal)
→ linker patches B's jal to: `jal x1, -260`

J-type offset: jal encodes a 20-bit signed PC-relative offset — supporting ±1 MB jumps without knowing absolute addresses.

Stage 4 — Loader

The loader is part of the operating system. Loading is one of the main OS kernel tasks.

1. **Read executable header**
   Determine sizes of text and data segments.

2. **Create address space**
   OS allocates virtual memory regions for text, data, stack, and heap.

3. **Copy code and data**
   Copy text segment and static data from disk into the new address space.

4. **Push arguments onto stack**
   argc, argv, and environment variables placed at the top of the stack.

5. **Initialise registers**
   Clear general-purpose registers. Set sp to the top of the stack segment.

6. **Jump to startup routine**
   Move arguments from stack to a0/a1 (argc/argv). Set PC to main(). On main() return, call the exit syscall.

Virtual memory: Relocating loaders place the program at any virtual address — address-space layout randomisation (ASLR) exploits this for security.

KEY TAKEAWAYS

1 **ISA is the HW/SW contract**

The ISA defines what SW can ask HW to execute - independently of the μ-architecture implementation

2 **Four RISC-V design principles**

Regularity, Common Case Fast, Smaller is Faster, Good Compromises

3 **Load-store architecture**

All computation happens in registers. Only lw/sw touch memory. More IC but consistent CPIs, simpler HW.

4 **Six fixed-width encoding formats**

R/I/S/B/U/J — all 32 bits wide. Field positions are reused across formats to minimize decoder hardware.

5 **RISC: do less, do it simply**

No `subi`*, no* `not`*, no* `ble`*. Compilers synthesize complex operations from simple primitives.*

Next Lecture: Programming with RV32I — P&H sections 2.7, 2.8, 2.10, 2.12, 2.14